

# [Taller 03] series de Taylor y polinomios de Lagrange

Richard Tipantiza

## Tabla de Contenidos

|                                                                                                 |          |
|-------------------------------------------------------------------------------------------------|----------|
| <b>Taller 03: series de Taylor y polinomios de Lagrange</b>                                     | <b>1</b> |
| Graficar $f(x) = \ln(x)$ , $x_0 = 1$ y sus polinomios de orden 1, 2 y 3. . . . .                | 1        |
| $x_0 = 10$ . . . . .                                                                            | 3        |
| <b>Grafique las curvas de las series de Taylor de varios órdenes para los siguientes casos:</b> | <b>5</b> |
| $\cos(x)$ , $x_0 = 0$ . . . . .                                                                 | 5        |
| $\frac{1}{1-x}$ , $x_0 = 0$ . . . . .                                                           | 7        |
| <b>Encuentre el polinomio de Lagrange para los siguientes datos y grafique:</b>                 | <b>9</b> |
| $(0, 0), (30, 0.5), (60, \frac{\sqrt{3}}{2}), (90, 1)$ . . . . .                                | 9        |
| $(1, 1), (2, 2), (3, 2)$ . . . . .                                                              | 11       |
| $(-2, 5), (1, 7), (3, 11), (7, 34)$ . . . . .                                                   | 13       |

## Taller 03: series de Taylor y polinomios de Lagrange

**Graficar  $f(x) = \ln(x)$ ,  $x_0 = 1$  y sus polinomios de orden 1, 2 y 3.**

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x - 1

def g(x):
    return ((x - 1)**2 / 2) + x - 1
```

```

def h(x):
    return (1/3) * (x - 1) - ((x - 1)**2 / 2) + x - 1

def p(x):
    return np.log(x)

x_general = np.linspace(-1, 5, 400)
x_p = np.linspace(0.01, 5, 400)

y_f = f(x_general)
y_g = g(x_general)
y_h = h(x_general)
y_p = p(x_p)

plt.figure(figsize=(10, 8))

plt.plot(x_general, y_f, label='f(x) = x - 1')
plt.plot(x_general, y_g, label='g(x) = (x-1)2/2 + x - 1')
plt.plot(x_general, y_h, label='h(x) = 1/3(x-1) - (x-1)2/2 + x - 1')
plt.plot(x_p, y_p, label='p(x) = ln(x)', linestyle='--')

plt.title('Gráfica de Funciones (Zoom)')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)

plt.legend()

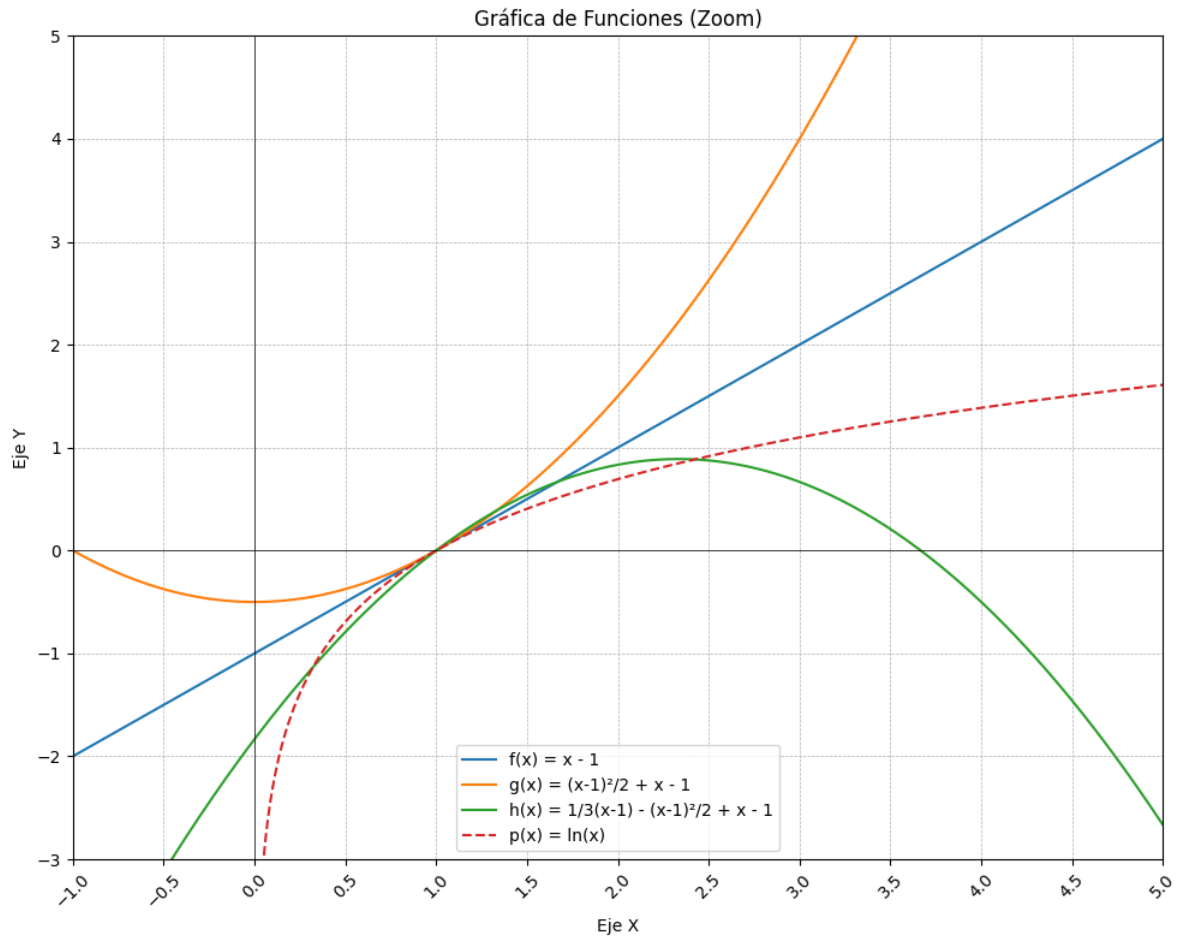
plt.xlim(-1, 5)
plt.ylim(-3, 5)

plt.xticks(np.arange(-1, 5.5, 0.5))
plt.yticks(np.arange(-3, 6, 1))

plt.xticks(rotation=45)
plt.tight_layout()

plt.savefig('funciones_con_zoom.png')

```



$$x_0 = 10$$

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return (900*x)/3000 - (5500)/3000 + np.log(10)

def g(x):
    return -45*x**2 / 3000 + (900*x)/3000 - (5500)/3000 + np.log(10)

def h(x):
    return (x**3 - 45*x**2 + 900*x - 5500)/3000 + np.log(10)
```

```

def p(x):
    return np.log(x)

x = np.linspace(0.01, 20, 500)

y_f = f(x)
y_g = g(x)
y_h = h(x)
y_p = p(x)

plt.figure(figsize=(10, 8))

# --- CORRECCIÓN AQUÍ ---
# He actualizado las etiquetas (label) para que muestren los nombres
# correctos de las funciones que estás graficando.
plt.plot(x, y_f, label='f(x) - Orden 1')
plt.plot(x, y_g, label='g(x) - Orden 2')
plt.plot(x, y_h, label='h(x) - Orden 3')
plt.plot(x, y_p, label='p(x) = ln(x)', linestyle='--', color='purple')

plt.title('Gráfica de Funciones (Punto de Interés x=10)')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(x=10, color='red', linestyle=':', label='x_o = 10')
plt.grid(True, which='both', linestyle='--', linewidth=0.5)

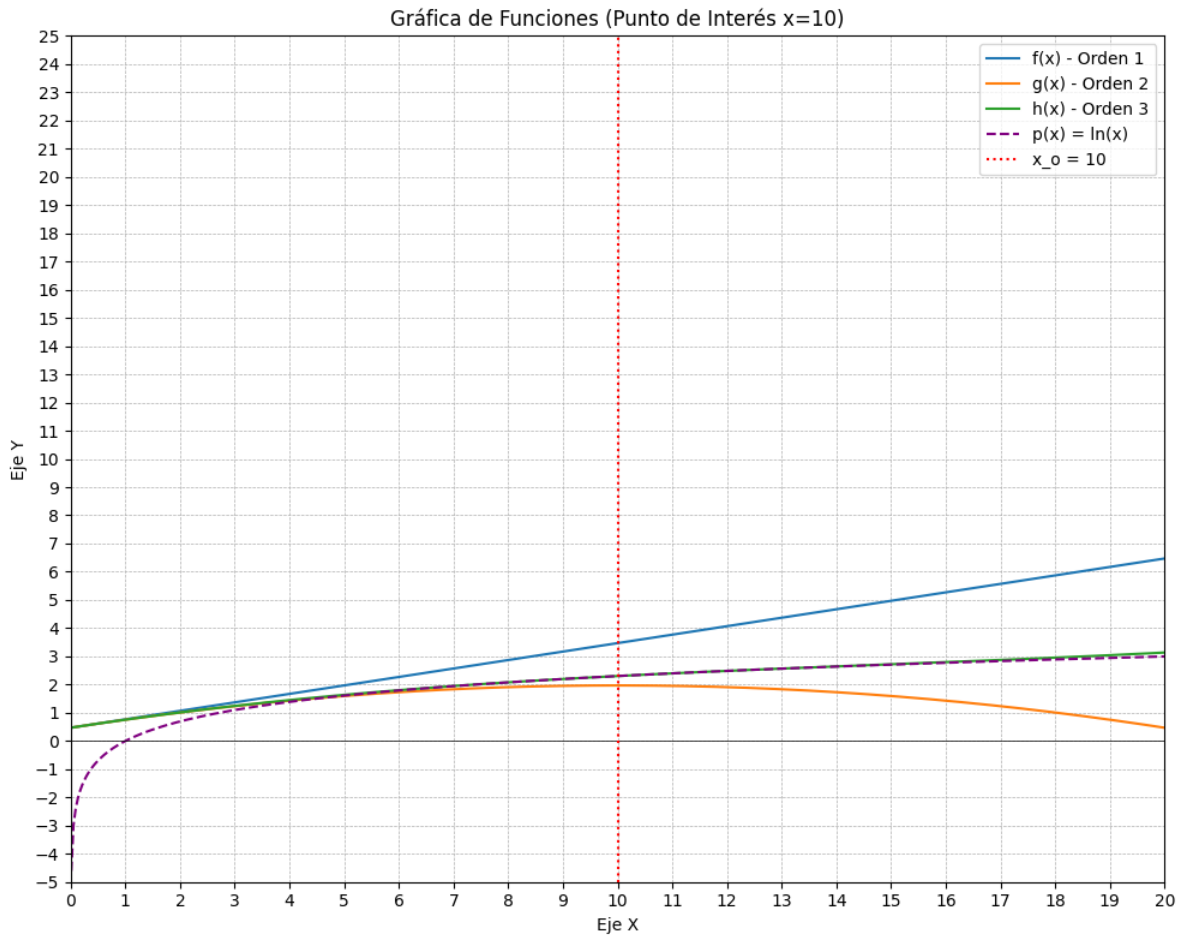
plt.xlim(0, 20)
plt.ylim(-5, 25)

plt.xticks(np.arange(0, 21, 1))
plt.yticks(np.arange(-5, 26, 1))

plt.legend()
plt.tight_layout()

plt.show()

```



**Grafique las curvas de las series de Taylor de varios órdenes para los siguientes casos:**

$$\cos(x), x_0 = 0$$

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.cos(x)

def g(x):
```

```

    return 1 - (x**2) / 2

def h(x):
    return 1 - (x**2) / 2 + (x**4) / 24

x_general = np.linspace(-np.pi, np.pi, 400)

y_f = f(x_general)
y_g = g(x_general)
y_h = h(x_general)

plt.figure(figsize=(10, 8))

plt.plot(x_general, y_f, label='f(x) = cos(x)', linewidth=2.5)
plt.plot(x_general, y_g, label='g(x) = 1 - x2/2 (Aproximación)', linestyle='--')
plt.plot(x_general, y_h, label='h(x) = 1 - x2/2 + x /24 (Aproximación)', linestyle=':')

plt.title('Aproximación de cos(x) en x0 = 0')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='red', linestyle=':', linewidth=2, label='x0 = 0')

plt.grid(True, which='both', linestyle='--', linewidth=0.5)

plt.legend()

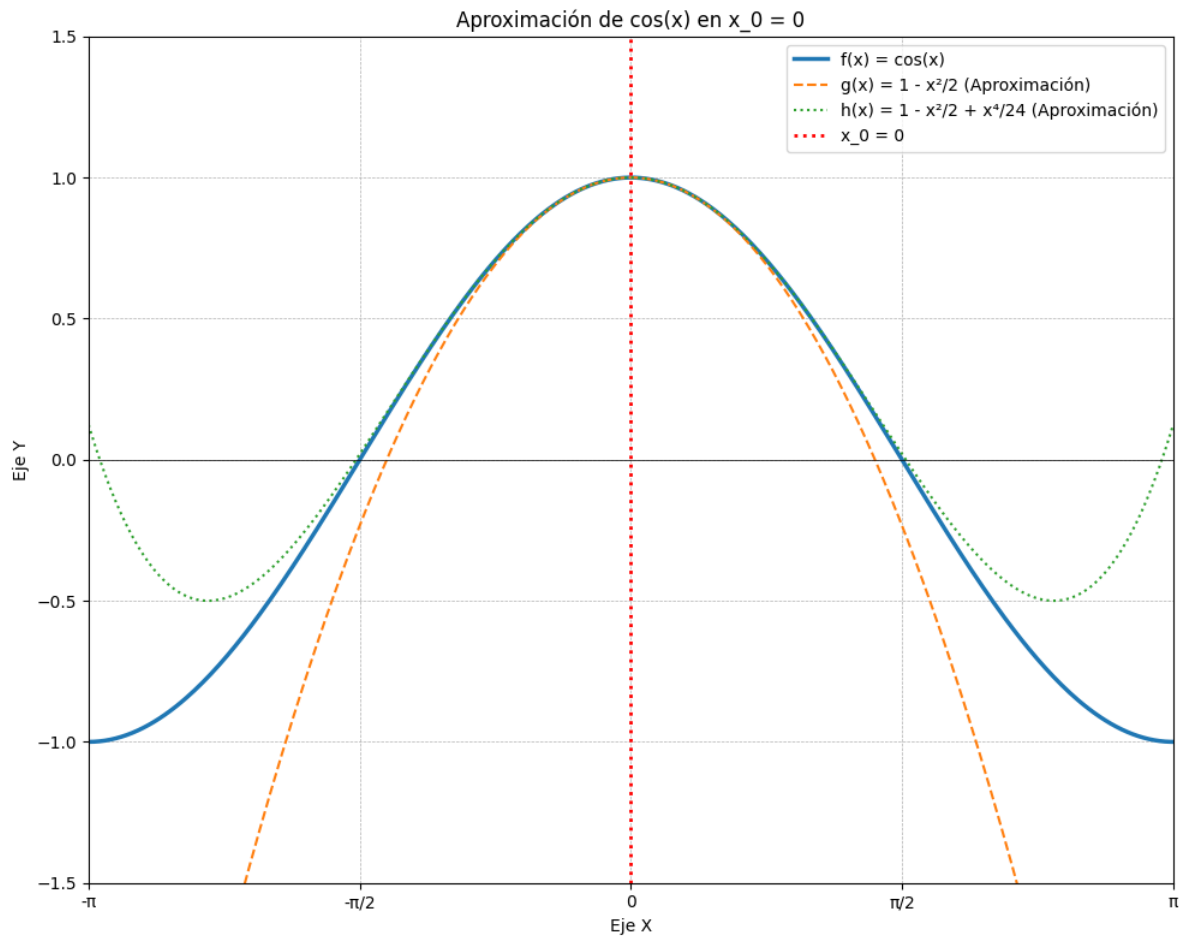
plt.xlim(-np.pi, np.pi)
plt.ylim(-1.5, 1.5)

plt.xticks(np.arange(-np.pi, np.pi + 0.5, np.pi/2),
           labels=['-', '- /2', '0', ' /2', '+'])
plt.yticks(np.arange(-1.5, 2.0, 0.5))

plt.tight_layout()

plt.savefig('cos_aproximacion.png')

```



$$\frac{1}{1-x}, x_0 = 0$$

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return 1/(1 - x)

def g(x):
    return 1 + x

def h(x):
    return 1 + x + x**2
```

```

def p(x):
    return 1 + x + x**2 + x**3

def q(x):
    return 1 + x + x**2 + x**3 + x**4

x_general = np.linspace(-2.5, 2.5, 500)

y_f = f(x_general)
y_f[np.abs(x_general - 1) < 0.05] = np.nan

y_g = g(x_general)
y_h = h(x_general)
y_p = p(x_general)
y_q = q(x_general)

plt.figure(figsize=(10, 8))

plt.plot(x_general, y_f, label='f(x) = 1/(1 - x)', linewidth=3, color='black')
plt.plot(x_general, y_g, label='g(x) = 1 + x', linestyle='--')
plt.plot(x_general, y_h, label='h(x) = 1 + x + x2', linestyle='--')
plt.plot(x_general, y_p, label='p(x) = 1 + x + x2 + x3', linestyle='--')
plt.plot(x_general, y_q, label='q(x) = 1 + x + x2 + x3 + x', linestyle='-', linewidth=2)

plt.title('Aproximación de 1/(1 - x)')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='red', linestyle=':', linewidth=1, label='x_0 = 0')

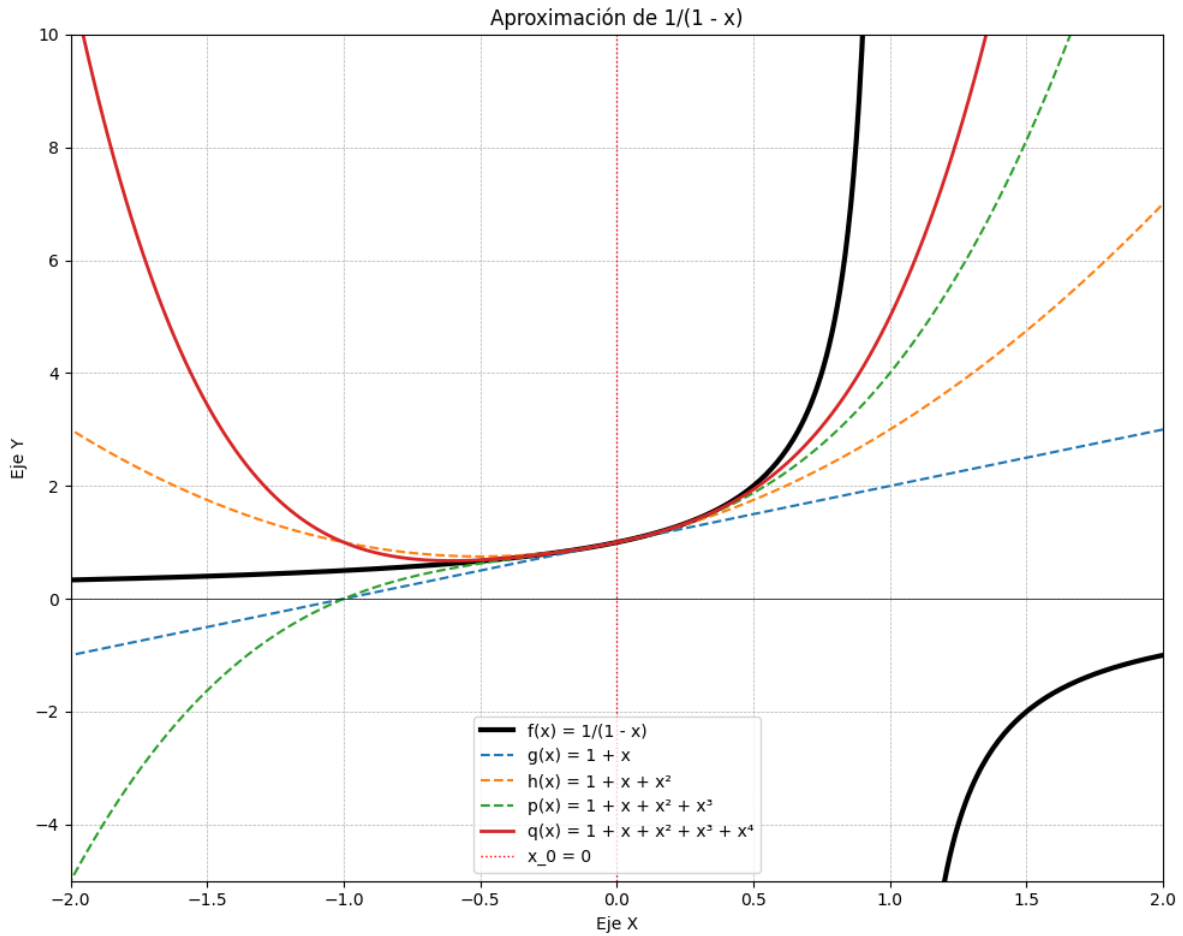
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.legend()

plt.xlim(-2, 2)
plt.ylim(-5, 10)

plt.tight_layout()
plt.savefig('geometrica_clean.png')
plt.show()

```





**Encuentre el polinomio de Lagrange para los siguientes datos y grafique:**

$(0, 0), (30, 0.5), (60, \frac{\sqrt{3}}{2}), (90, 1)$

```
import numpy as np
import matplotlib.pyplot as plt

def P(x):

    termino2 = (x**3 - 150*x**2 + 5400*x) / 108000
    termino3 = (np.sqrt(3) * (x**3 - 120*x**2 + 2700*x)) / -108000
```

```

    termino4 = ( (x**3 - 90*x**2 + 1800*x) ) / 162000

    return termino2 + termino3 + termino4

x_grafica = np.linspace(-5, 95, 200)

y_grafica = P(x_grafica)

plt.figure(figsize=(10, 6))

plt.plot(x_grafica, y_grafica, label='Polinomio de Lagrange', color='blue')

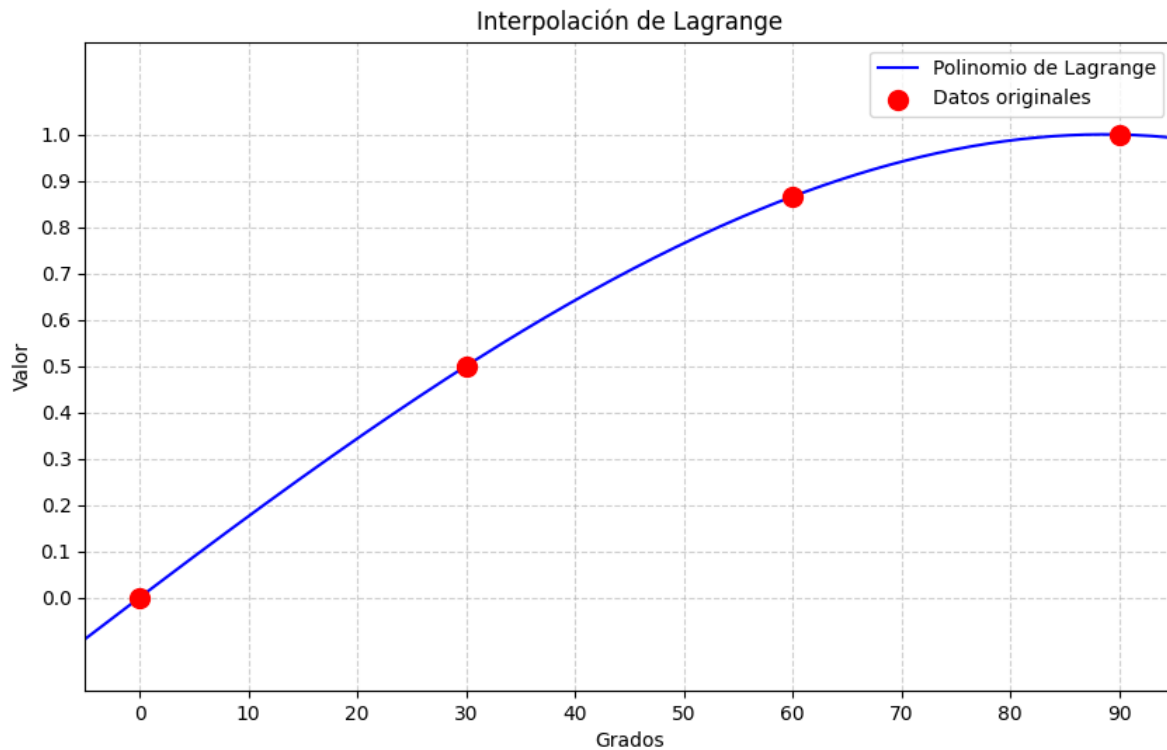
puntos_x = [0, 30, 60, 90]
puntos_y = [0, 0.5, np.sqrt(3)/2, 1]
plt.scatter(puntos_x, puntos_y, color='red', s=100, label='Datos originales', zorder=5)

plt.title('Interpolación de Lagrange')
plt.xlabel('Grados')
plt.ylabel('Valor')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.xlim(-5, 95)
plt.ylim(-0.2, 1.2)
plt.xticks(np.arange(0, 96, 10))
plt.yticks(np.arange(0, 1.1, 0.1))

plt.show()

```



$(1, 1), (2, 2), (3, 2)$

```
import numpy as np
import matplotlib.pyplot as plt

def P(x):
    termino1 = (x**2 - 5*x + 6) / 2
    termino2 = -2*x**2 + 8*x - 6
    termino3 = x**2 - 3*x + 2

    return termino1 + termino2 + termino3

x_grafica = np.linspace(0, 4, 100)
y_grafica = P(x_grafica)

plt.figure(figsize=(10, 6))
```

```

plt.plot(x_grafica, y_grafica, label='Polinomio de Lagrange', color='blue')

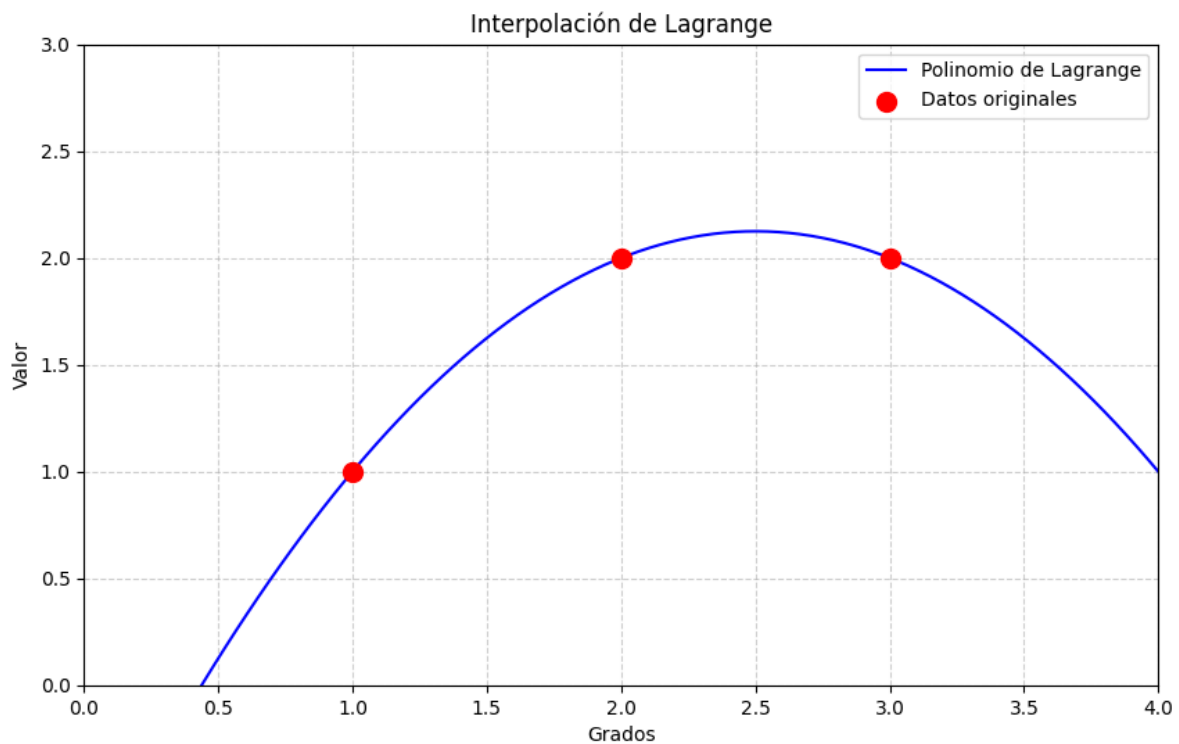
puntos_x = [1, 2, 3]
puntos_y = [1, 2, 2]
plt.scatter(puntos_x, puntos_y, color='red', s=100, label='Datos originales', zorder=5)

plt.title('Interpolación de Lagrange')
plt.xlabel('Grados')
plt.ylabel('Valor')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.xlim(0, 4)
plt.ylim(0, 3)

plt.show()

```



$(-2, 5), (1, 7), (3, 11), (7, 34)$

```
import numpy as np
import matplotlib.pyplot as plt

def P(x):
    termino1 = (5*x**3 - 55*x**2 + 155*x - 105) / -135
    termino2 = (7*x**3 - 56*x**2 + 7*x + 294) / 36
    termino3 = (11*x**3 - 66*x**2 - 99*x + 154) / -40
    termino4 = (34*x**3 - 68*x**2 - 170*x + 204) / 216

    return termino1 + termino2 + termino3 + termino4

x_grafica = np.linspace(-3, 8, 100)

y_grafica = P(x_grafica)

plt.figure(figsize=(10, 6))

plt.plot(x_grafica, y_grafica, label='Polinomio de Lagrange', color='blue')

puntos_x = [-2, 1, 3, 7]
puntos_y = [5, 7, 11, 34]
plt.scatter(puntos_x, puntos_y, color='red', s=100, label='Datos originales', zorder=5)

plt.title('Interpolación de Lagrange')
plt.xlabel('Grados')
plt.ylabel('Valor')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.xlim(-3, 8)
plt.ylim(0, 40)

plt.show()
```

